

Atividade: Condição de Disputa

Aula 03, Interação entre Tarefas (Parte II)

Vinicius Guerra
IFSC, Campus São José
Eng. de Telecomunicações

Maio de 2026

Enunciado

O slide final da aula pede para aplicar três algoritmos de exclusão mútua ao código de depósito bancário concorrente. O código tem uma condição de disputa porque a leitura e a escrita do `saldo` acontecem em momentos diferentes:

1. Alternância de Uso (*strict turn-taking*)
2. Algoritmo de Peterson (1981)
3. TSL, *Test-and-Set Lock* (operação atômica)

Em todos os testes uso `saldo=100`, `v1=50` e `v2=30`. O resultado correto é 180.

1 Demonstração da Condição de Disputa

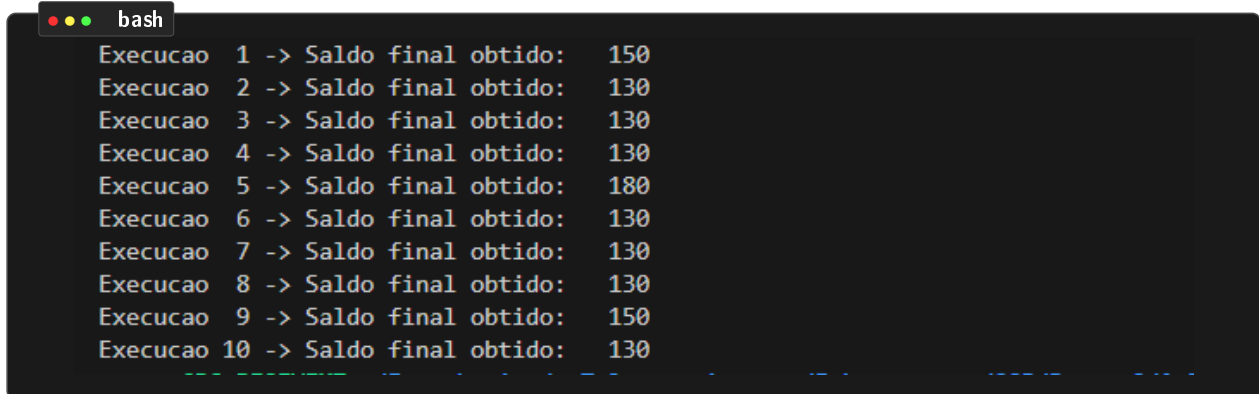
O código original não tem nenhum mecanismo de exclusão mútua. As três operações (ler, somar, escrever) das duas *threads* se intercalam e um dos depósitos acaba sendo perdido. O `usleep(100)` entre a leitura e a escrita aumenta a chance de troca de contexto e ajuda a expor o problema.

```
1  /*
2   * 0_problema.c
3   * Código original da atividade (slide 39).
4   * Demonstra a condicao de disputa: o saldo final NAO eh sempre 180.
5   *
6   * Compilar: gcc -Wall -O0 -pthread 0_problema.c -o 0_problema
7   * Executar: ./0_problema
8   */
9  #include <stdio.h>
10 #include <unistd.h>
11 #include <pthread.h>
12
13 int saldo = 100;
14
15 void* depositar(void* arg) {
16     int valor = *(int*)arg;
17     int temp = saldo;    /* Passo 1: Leitura */
18
19     /* Forca a troca de contexto entre leitura e escrita,
20      expondo a condicao de disputa. */
21     usleep(100);
22
23     temp += valor;        /* Passo 2: Soma */
24     saldo = temp;        /* Passo 3: Escrita */
25     return NULL;
26 }
27
28 int main(void) {
```

```
29 pthread_t t1, t2;  
30 int v1 = 50, v2 = 30;  
31  
32 pthread_create(&t1, NULL, depositar, &v1);  
33 pthread_create(&t2, NULL, depositar, &v2);  
34  
35 pthread_join(t1, NULL);  
36 pthread_join(t2, NULL);  
37  
38 printf("Saldo final esperado: 180\n");  
39 printf("Saldo final obtido: %d\n", saldo);  
40 return 0;  
41 }
```

Listing 1: 0_problema.c: código original com condição de disputa

Captura de tela



```
bash  
Execucao 1 -> Saldo final obtido: 150  
Execucao 2 -> Saldo final obtido: 130  
Execucao 3 -> Saldo final obtido: 130  
Execucao 4 -> Saldo final obtido: 130  
Execucao 5 -> Saldo final obtido: 180  
Execucao 6 -> Saldo final obtido: 130  
Execucao 7 -> Saldo final obtido: 130  
Execucao 8 -> Saldo final obtido: 130  
Execucao 9 -> Saldo final obtido: 150  
Execucao 10 -> Saldo final obtido: 130
```

Figura 1: 10 execuções do código original sem proteção.

O resultado é não-determinístico. Na maior parte das execuções o saldo final é 130 ou 150, e raramente 180 (quando o escalonador, por sorte, não intercala as operações). Quando t1 e t2 leem `saldo=100` antes de qualquer escrita, o último a gravar sobrescreve o depósito do outro e o sistema perde 50 ou 30 reais.

2 Solução 1: Alternância de Uso

Uma variável global `turn` indica de quem é a vez. A tarefa só entra na seção crítica quando `turn == task`, e ao sair passa o turno para a próxima.

```
1  /*
2   * 1_alternancia.c
3   * Solucao: Alternancia de Uso (strict turn-taking).
4   *
5   * Existe uma variavel global "turn" que indica qual tarefa
6   * pode entrar na secao critica. Cada tarefa so entra quando
7   * turn == task. Ao sair, passa o turno para a proxima.
8   *
9   * Compilar: gcc -Wall -O0 -pthread 1_alternancia.c -o 1_alternancia
10  * Executar: ./1_alternancia
11  */
12 #include <stdio.h>
13 #include <unistd.h>
14 #include <pthread.h>
15
16 #define NUM_TASKS 2
17
18 int saldo = 100;
19
20 /* --- mecanismo de exclusao mutua: alternancia de uso --- */
21 volatile int turn = 0; /* comeca pela tarefa 0 */
22
23 void enter(int task) {
24     while (turn != task) {
25         /* espera ocupada ate ser sua vez */
26     }
27 }
28
29 void leave(int task) {
30     (void)task;
31     turn = (turn + 1) % NUM_TASKS;
32 }
33
34 /* --- thread de deposito --- */
35 typedef struct {
36     int id;
37     int valor;
38 } arg_t;
39
40 void* depositar(void* arg) {
41     arg_t* a = (arg_t*)arg;
42
43     enter(a->id); /* entra na secao critica */
44     int temp = saldo; /* Passo 1: Leitura */
45     usleep(100); /* forca troca de contexto */
46     temp += a->valor; /* Passo 2: Soma */
47     saldo = temp; /* Passo 3: Escrita */
48     leave(a->id); /* libera a secao critica */
49
50     return NULL;
51 }
52
53 int main(void) {
54     pthread_t t1, t2;
55     arg_t a1 = {0, 50};
```

```
56     arg_t a2 = {1, 30};
57
58     pthread_create(&t1, NULL, depositar, &a1);
59     pthread_create(&t2, NULL, depositar, &a2);
60
61     pthread_join(t1, NULL);
62     pthread_join(t2, NULL);
63
64     printf("Saldo final esperado: 180\n");
65     printf("Saldo final obtido:    %d\n", saldo);
66     return 0;
67 }
```

Listing 2: 1_alternancia.c

Captura de tela

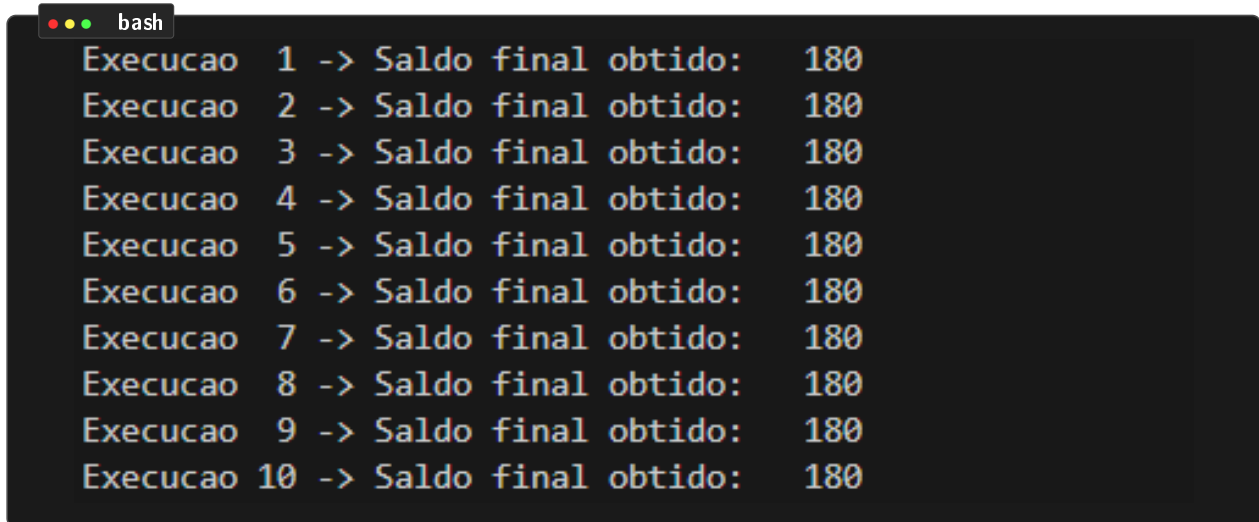


Figura 2: 10 execuções com alternância de uso. Saldo final sempre 180.

A solução resolve a disputa, mas cria uma dependência rígida entre as tarefas. Se a tarefa 0 não quiser entrar, a tarefa 1 fica esperando o turno indefinidamente. Por isso ela falha no critério de independência.

3 Solução 2: Algoritmo de Peterson

Cada tarefa sinaliza interesse (`wants[task]=1`) e cede o turno para a outra (`turn=other`). O `while` só bloqueia se a outra também quer entrar e for a vez dela.

```

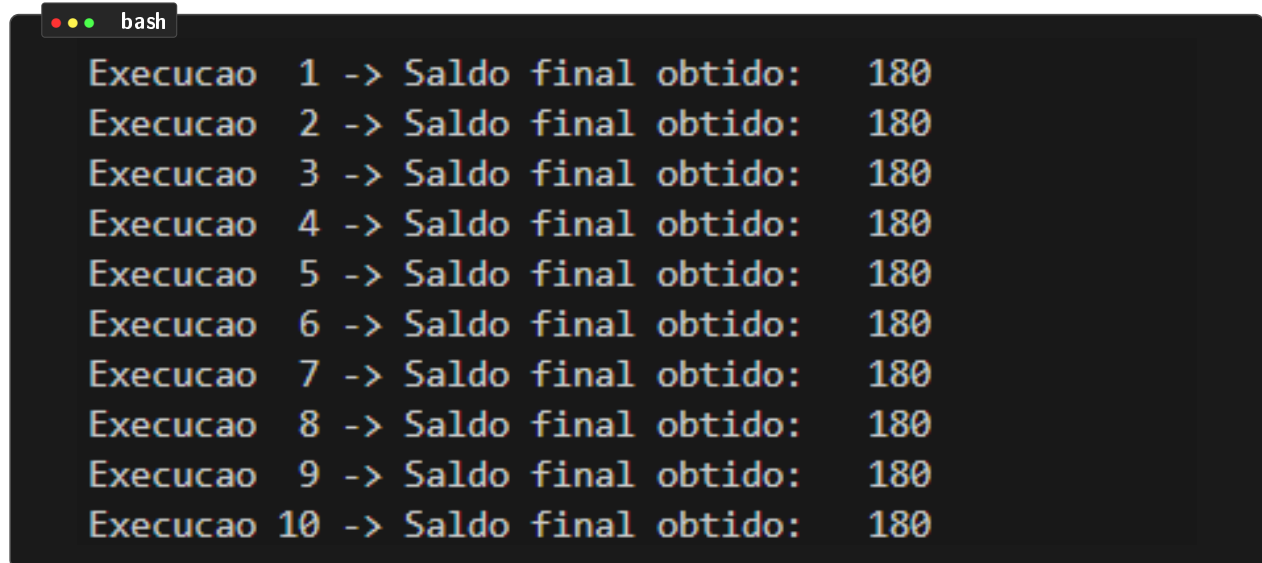
1  /*
2   * 2_peterson.c
3   * Solucao: Algoritmo de Peterson (1981) para 2 tarefas.
4   *
5   * Cada tarefa expressa o desejo de entrar (wants[task] = 1)
6   * e cede gentilmente a vez para a outra (turn = other).
7   * O while bloqueia somente se a outra tarefa quer entrar
8   * E for a vez dela. Garante exclusao mutua e espera limitada.
9   *
10  * Compilar: gcc -Wall -O0 -pthread 2_peterson.c -o 2_peterson
11  * Executar: ./2_peterson
12  */
13 #include <stdio.h>
14 #include <unistd.h>
15 #include <pthread.h>
16
17 int saldo = 100;
18
19 /* --- mecanismo de exclusao mutua: Peterson --- */
20 volatile int turn = 0;
21 volatile int wants[2] = {0, 0};
22
23 void enter(int task) {
24     int other = 1 - task;
25     wants[task] = 1;
26     turn = other;
27
28     /* Barreira para garantir ordem de visibilidade
29      das escritas em wants/turn antes da leitura. */
30     __sync_synchronize();
31
32     while (wants[other] && turn == other) {
33         /* espera ocupada */
34     }
35 }
36
37 void leave(int task) {
38     __sync_synchronize();
39     wants[task] = 0;
40 }
41
42 /* --- thread de deposito --- */
43 typedef struct {
44     int id;
45     int valor;
46 } arg_t;
47
48 void* depositar(void* arg) {
49     arg_t* a = (arg_t*)arg;
50
51     enter(a->id);
52     int temp = saldo;           /* Passo 1: Leitura */
53     usleep(100);               /* forca troca de contexto */
54     temp += a->valor;          /* Passo 2: Soma */
55     saldo = temp;              /* Passo 3: Escrita */

```

```
56     leave(a->id);
57
58     return NULL;
59 }
60
61 int main(void) {
62     pthread_t t1, t2;
63     arg_t a1 = {0, 50};
64     arg_t a2 = {1, 30};
65
66     pthread_create(&t1, NULL, depositar, &a1);
67     pthread_create(&t2, NULL, depositar, &a2);
68
69     pthread_join(t1, NULL);
70     pthread_join(t2, NULL);
71
72     printf("Saldo final esperado: 180\n");
73     printf("Saldo final obtido:    %d\n", saldo);
74     return 0;
75 }
```

Listing 3: 2_peterson.c

Captura de tela

*Figura 3: 10 execuções com Peterson. Saldo final sempre 180.*

Aqui o resultado também é correto e a independência fica preservada: se só uma tarefa quer entrar, ela entra imediatamente. As limitações são que o algoritmo só funciona para duas tarefas e mantém espera ocupada. Precisei usar `__sync_synchronize()` para impedir que o compilador e a CPU reordenassem as escritas em `wants` e `turn`, o que poderia quebrar a lógica do algoritmo.

4 Solução 3: TSL (Test-and-Set Lock)

A instrução TSL lê o valor antigo e escreve 1 num único passo atômico. No código uso a built-in `__sync_lock_test_and_set` do GCC, que gera a instrução equivalente no x86 (`XCHG` ou `LOCK CMPXCHG`). Funciona para qualquer número de tarefas e em multiprocessadores.

```

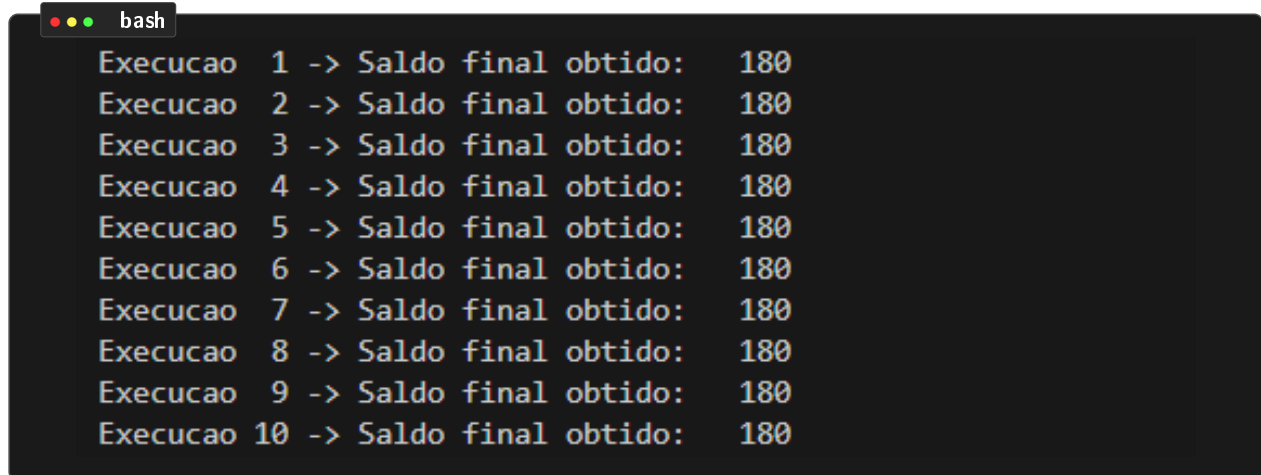
1  /*
2   * 3_tsl.c
3   * Solucao: Test-and-Set Lock (TSL) - operacao atomica.
4   *
5   * A instrucao TSL le o valor antigo e escreve 1 atomicamente,
6   * em um unico passo indivisivel. Se o valor antigo era 0, a
7   * tarefa entra na secao critica; se era 1, fica em espera ocupada.
8   *
9   * Aqui usamos a built-in do GCC __sync_lock_test_and_set, que
10  * gera a instrucao atomica equivalente (XCHG/LOCK CMPXCHG no x86).
11  *
12  * Compilar: gcc -Wall -O0 -pthread 3_tsl.c -o 3_tsl
13  * Executar: ./3_tsl
14  */
15 #include <stdio.h>
16 #include <unistd.h>
17 #include <pthread.h>
18
19 int saldo = 100;
20
21 /* --- mecanismo de exclusao mutua: TSL --- */
22 volatile int lock = 0;
23
24 /* TSL atomico: equivalente a:
25  old = *x; *x = 1; return old; (porem indivisivel) */
26 static inline int TSL(volatile int *x) {
27     return __sync_lock_test_and_set(x, 1);
28 }
29
30 void enter(volatile int *l) {
31     while (TSL(l)) {
32         /* espera ocupada: enquanto TSL retornar 1,
33          significa que a trava ja estava fechada. */
34     }
35 }
36
37 void leave(volatile int *l) {
38     __sync_lock_release(l); /* equivale a *l = 0 com barreira */
39 }
40
41 /* --- thread de deposito --- */
42 void* depositar(void* arg) {
43     int valor = *(int*)arg;
44
45     enter(&lock);
46     int temp = saldo;          /* Passo 1: Leitura */
47     usleep(100);              /* forca troca de contexto */
48     temp += valor;            /* Passo 2: Soma */
49     saldo = temp;             /* Passo 3: Escrita */
50     leave(&lock);
51
52     return NULL;
53 }
54

```

```
55 int main(void) {  
56     pthread_t t1, t2;  
57     int v1 = 50, v2 = 30;  
58  
59     pthread_create(&t1, NULL, depositar, &v1);  
60     pthread_create(&t2, NULL, depositar, &v2);  
61  
62     pthread_join(t1, NULL);  
63     pthread_join(t2, NULL);  
64  
65     printf("Saldo final esperado: 180\n");  
66     printf("Saldo final obtido:    %d\n", saldo);  
67     return 0;  
68 }
```

Listing 4: 3_tsl.c

Captura de tela

*Figura 4: 10 execuções com TSL. Saldo final sempre 180.*

Resolve a disputa, vale para N tarefas e roda em sistemas com vários núcleos. O único ponto fraco é a espera ocupada. É a base usada pelos mutexes reais do Linux (`pthread_mutex_t`).

Conclusão

Sem nenhum mecanismo de proteção, três operações simples (ler, somar e escrever) sobre uma variável compartilhada produzem resultados imprevisíveis: nas 10 execuções tive 130, 150 e ocasionalmente 180. As três soluções vistas em aula corrigem o problema e o saldo final passa a ser sempre 180.

Cada uma tem limitações próprias. A alternância é rígida, o Peterson só serve para duas tarefas, e todas usam espera ocupada. Por isso, na prática usa-se mecanismos mais robustos como mutexes e semáforos.